DS Homework 5

NAME: Manoj Kumar M
ROLL NO:-2025115304

1. Use linked list for polynomial representation and perform addition of two polynomials.

Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int coeff, power;
   struct Node *next;
};

struct Node* create(int c, int p) {
   struct Node *n = malloc(sizeof(struct Node));
   n->coeff = c;
   n->power = p;
   n->next = NULL;
   return n;
}

struct Node* add(struct Node *a, struct Node *b) {
   struct Node *r = NULL, *t, **p = &r;

   while (a || b) {
      int c, pow;

      if (!b || (a && a->power > b->power)) {
         c = a->coeff;
         pow = a->power;
         a = a->next;
      }
      else if (!a || b->power > a->power) {
         c = b->coeff;
         pow = b->power;
         b = b->next;
      }
      else {
         c = a->coeff + b->coeff;
         pow = a->power;
         a = a->next;
         b = b->next;
      }
```

```
        t = create(c, pow);
        *p = t;
        p = &t->next;
    }
    return r;
}

void display(struct Node *p) {
    while (p) {
        printf("%dx^%d", p->coeff, p->power);
        if (p->next) printf(" + ");
        p = p->next;
    }
}

int main() {
    struct Node *a = create(3,2);
    a->next = create(2,1);
    a->next->next = create(1,0);

    struct Node *b = create(4,2);
    b->next = create(3,1);
    b->next->next = create(2,0);

    struct Node *r = add(a,b);

    printf("Result: ");
    display(r);

    return 0;
}
```

Output

```
Result: 7x^2 + 5x^1 + 3x^0
```

2. Find previous node of circular LL.

Code

```
#include <stdio.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node a = {10, NULL};
    struct Node b = {20, NULL};
    struct Node c = {30, NULL};

    a.next = &b;
    b.next = &c;
```

```
    c.next = &a;

    int key = 30;
    struct Node *p = &a;

    do {
        if (p->next->data == key) {
            printf("Previous node = %d", p->data);
            break;
        }
        p = p->next;
    } while (p != &a);

    return 0;
}
```

Output

```
Previous node = 20
```

3. Insert at a given position of a circular LL.

Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *head, *a, *b, *c, *n, *p;

    a = malloc(sizeof(struct Node));
    b = malloc(sizeof(struct Node));
    c = malloc(sizeof(struct Node));

    a->data = 10;
    b->data = 20;
    c->data = 30;

    a->next = b;
    b->next = c;
    c->next = a;
    head = a;

    n = malloc(sizeof(struct Node));
    n->data = 15;

    p = head;
    p = p->next;
    n->next = p->next;
```

```
    p->next = n;

    p = head;
    do {
        printf("%d ", p->data);
        p = p->next;
    } while (p != head);

    return 0;
}
```

Output

```
10 20 15 30
```

4. Find largest 2nd element in a Queue.

Code

```
#include <stdio.h>

int main() {
    int q[] = {10, 40, 20, 50, 30};
    int n = 5;
    int largest = -9999, second = -9999;

    for (int i = 0; i < n; i++) {
        if (q[i] > largest) {
            second = largest;
            largest = q[i];
        }
        else if (q[i] > second && q[i] != largest) {
            second = q[i];
        }
    }

    printf("Second largest = %d", second);

    return 0;
}
```

Output

```
Second largest = 40
```

5. Insert 3rd element and check whether 1st and 3rd elements are same in a LL.

Code

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
   int data;
   struct Node *next;
};

int main() {
   struct Node *head, *a, *b, *c, *n;

   a = malloc(sizeof(struct Node));
   b = malloc(sizeof(struct Node));
   c = malloc(sizeof(struct Node));

   a->data = 10;
   b->data = 20;
   c->data = 30;

   a->next = b;
   b->next = c;
   c->next = NULL;
   head = a;

   n = malloc(sizeof(struct Node));
   n->data = 10;

   n->next = head->next->next;
   head->next->next = n;

   if (head->data == head->next->next->data)
      printf("1st and 3rd elements are same");
   else
      printf("1st and 3rd elements are not same");

   return 0;
}
```

Output

```
1st and 3rd elements are same
```

6. Implement two stacks in one array. Write a routine to traverse and display.

Code

```
#include <stdio.h>

#define MAX 10

int a[MAX];
int top1 = -1, top2 = MAX;

void push1(int x) {
   if (top1 + 1 < top2)
      a[++top1] = x;
}
```

```
void push2(int x) {
   if (top1 + 1 < top2)
      a[--top2] = x;
}

void display() {
   int i;

   printf("Stack 1: ");
   for (i = 0; i <= top1; i++)
      printf("%d ", a[i]);

   printf("\nStack 2: ");
   for (i = top2; i < MAX; i++)
      printf("%d ", a[i]);
}

int main() {
   push1(10);
   push1(20);

   push2(50);
   push2(40);

   display();

   return 0;
}
```

Output

```
Stack 1: 10 20
Stack 2: 40 50
```

7. Given a linked list containing n head nodes where every node in the LL contains two pointers — vertical pointer to next node in the sub-linked list. The head nodes are sorted in ascending order based on their data. We need to flatten the LL so that all the nodes appear in a single LL while maintaining sorted order.

Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
   struct Node *down;
};

struct Node* createNode(int data)
{
   struct Node *newNode =
      (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = data;
    newNode->next = NULL;
    newNode->down = NULL;

    return newNode;
}

struct Node* merge(struct Node *a, struct Node *b)
{
    struct Node dummy;
    struct Node *temp = &dummy;

    dummy.down = NULL;

    while (a != NULL && b != NULL)
    {
        if (a->data <= b->data)
        {
            temp->down = a;
            a = a->down;
        }
        else
        {
            temp->down = b;
            b = b->down;
        }

        temp = temp->down;
    }

    if (a != NULL)
        temp->down = a;
    else
        temp->down = b;

    return dummy.down;
}

struct Node* flatten(struct Node *head)
{
    if (head == NULL || head->next == NULL)
        return head;

    head->next = flatten(head->next);

    head = merge(head, head->next);

    return head;
}

void display(struct Node *head)
{
    while (head != NULL)
    {
        printf("%d", head->data);
```

```
        if (head->down != NULL)
            printf(" -> ");

        head = head->down;
    }

    printf("\n");
}

int main()
{
    struct Node *head;

    head = createNode(5);
    head->next = createNode(10);
    head->next->next = createNode(19);
    head->next->next->next = createNode(28);

    head->down = createNode(7);
    head->down->down = createNode(8);

    head->next->down = createNode(20);

    head->next->next->down = createNode(22);

    head->next->next->next->down = createNode(40);
    head->next->next->next->down->down = createNode(45);

    head = flatten(head);

    printf("Flattened Linked List:\n");
    display(head);

    return 0;
}
```

Output

```
Flattened Linked List:
5 -> 7 -> 8 -> 10 -> 19 -> 20 -> 22 -> 28 -> 40 -> 45
```

8. Given k sorted LL of different sizes, we need to merge them into a single LL for maintaining their sorted order.

Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node* merge(struct Node *a, struct Node *b) {
```

```
    struct Node dummy;
    struct Node *t = &dummy;

    dummy.next = NULL;

    while (a && b) {
        if (a->data < b->data) {
            t->next = a;
            a = a->next;
        }
        else {
            t->next = b;
            b = b->next;
        }
        t = t->next;
    }

    t->next = a ? a : b;
    return dummy.next;
}

int main() {
    struct Node *a, *b, *c, *r;

    a = malloc(sizeof(struct Node));
    b = malloc(sizeof(struct Node));
    c = malloc(sizeof(struct Node));

    a->data = 10;
    a->next = NULL;

    b->data = 20;
    b->next = NULL;

    c->data = 30;
    c->next = NULL;

    r = merge(a, b);
    r = merge(r, c);

    printf("Merged list: ");

    while (r) {
        printf("%d ", r->data);
        r = r->next;
    }

    return 0;
}
```

Output

```
Merged list: 10 20 30
```

9. Swap kth node from beginning with kth node from end in a LL.

Code

```
#include <stdio.h>

struct Node {
   int data;
   struct Node *next;
};

int main() {
   struct Node a = {10, NULL};
   struct Node b = {20, NULL};
   struct Node c = {30, NULL};
   struct Node d = {40, NULL};
   struct Node e = {50, NULL};

   a.next = &b;
   b.next = &c;
   c.next = &d;
   d.next = &e;

   struct Node *p = &a;
   struct Node *q = &a;
   int k = 2, n = 0;

   while (p) {
      n++;
      p = p->next;
   }

   p = &a;

   for (int i = 1; i < k; i++)
      p = p->next;

   q = &a;

   for (int i = 1; i <= n-k; i++)
      q = q->next;

   int temp = p->data;
   p->data = q->data;
   q->data = temp;

   p = &a;

   printf("After swapping: ");

   while (p) {
      printf("%d ", p->data);
      p = p->next;
   }
```

```
    return 0;
}
```

Output

```
After swapping: 10 40 30 20 50
```

10. Multiply two numbers represented by two LL.

Code

```
#include <stdio.h>

struct Node {
    int data;
    struct Node *next;
};

long long value(struct Node *p) {
    long long n = 0;

    while (p) {
        n = n * 10 + p->data;
        p = p->next;
    }

    return n;
}

int main() {
    struct Node a = {1, NULL};
    struct Node b = {2, NULL};

    struct Node c = {2, NULL};
    struct Node d = {0, NULL};
    struct Node e = {0, NULL};

    a.next = &b;

    c.next = &d;
    d.next = &e;

    printf("First number = %lld\n", value(&a));
    printf("Second number = %lld\n", value(&c));
    printf("Product = %lld", value(&a) * value(&c));

    return 0;
}
```

Output

```
First number = 12
Second number = 200
Product = 2400
```